Objectives: Learn how to call standard Math class methods and use their results in complete programs.

Overview:

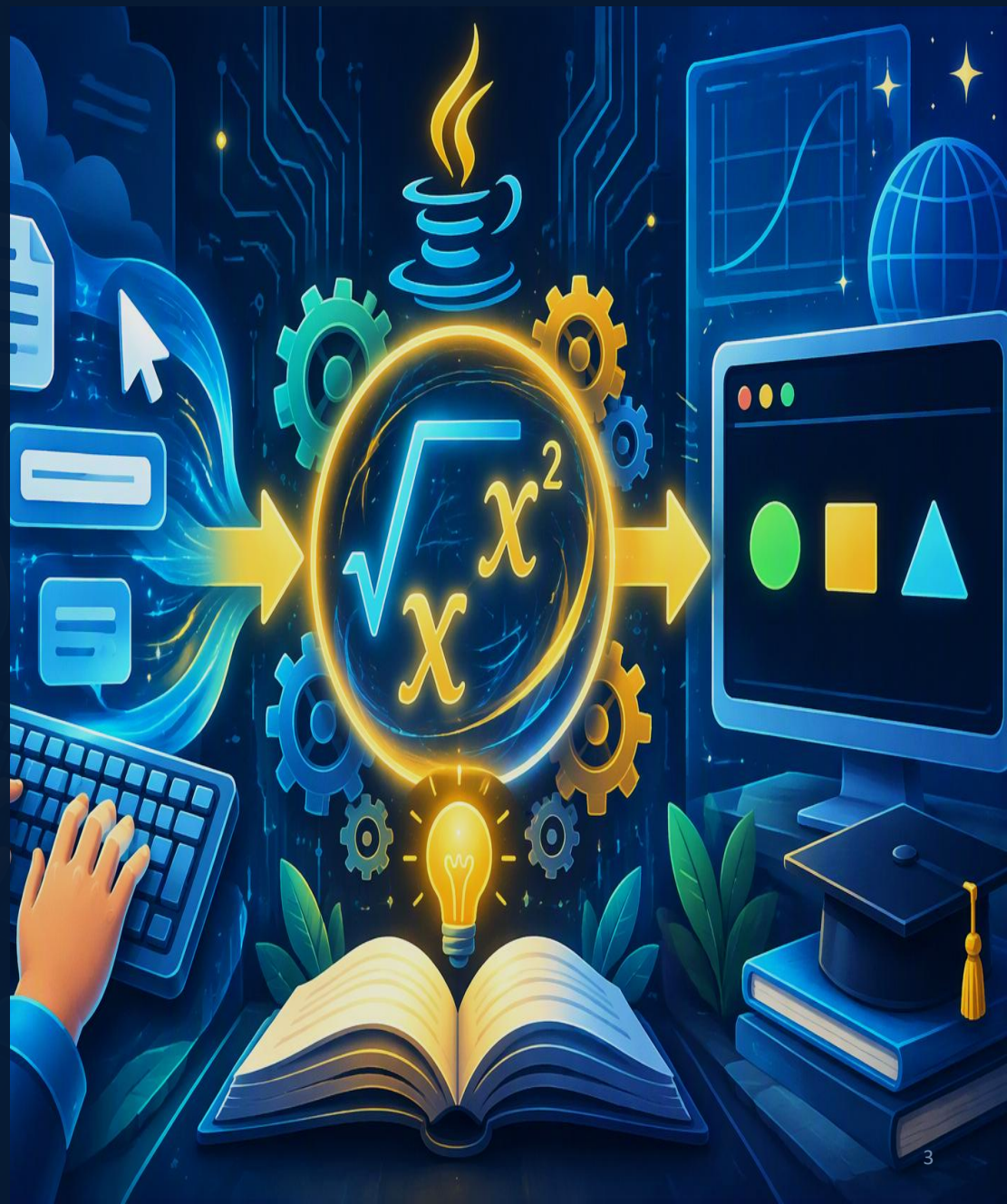
- Basic operators in math
- Java Anatomy for Math Method Call
- Java's π constant
- Examples

2.10

Using Math Methods

Beyond +, −, ×, and ÷ in Java

Learn how to call standard Math class methods and use their results in complete programs.



Why do we need Math methods?

Some programs need operations that basic arithmetic operators do not provide directly.

Basic operators

+	−	×	÷

But what if your program needs a square root, a power, or an absolute value?

Java provides these operations through methods in the Math class.



The Math class

A standard Java class that provides many mathematical operations.

Key idea

- The Math class contains many math methods.
- It is part of Java's standard language package.
- **No import statement is required to use Math.**

```
double root = Math.sqrt(25.0);
```

Read it as: "***Call the sqrt method*** **from the Math class.**"

No import needed

What is a method?

A method is a named group of statements that performs a task.



Calling a method means invoking its name so Java executes those statements.

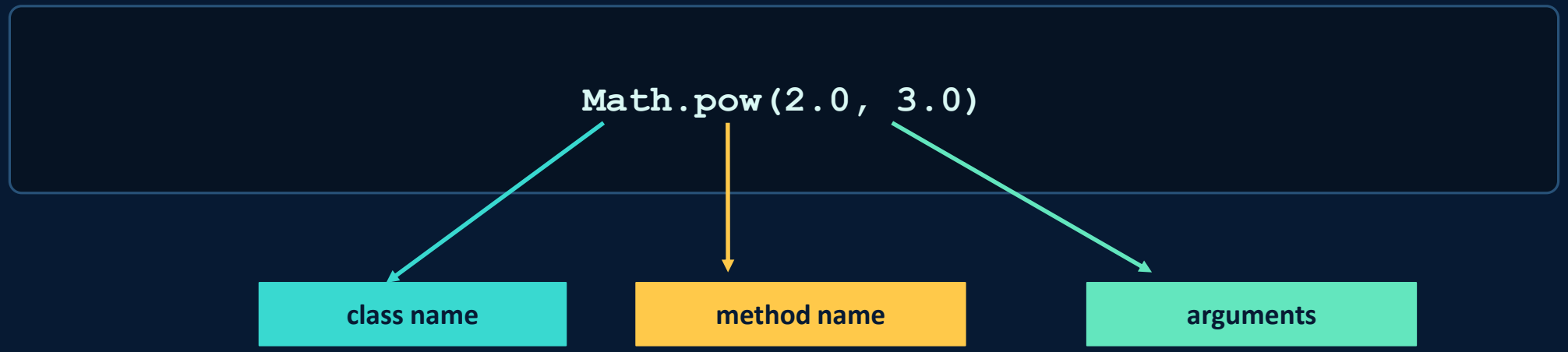
```
Math.sqrt(81.0)
```

This expression is a method call.

The method performs its operation and produces a result that your program can use.

Anatomy of a Math method call

Use Math. before the method name. Put input values inside parentheses.



If there is more than one argument, separate the values with commas.

Common examples #1

A few common methods from the standard Math class.

Method	Behavior	Example
sqrt(x)	Square root of x	Math.sqrt(9.0) evaluates to 3.0.
pow(x, y)	Power: x^y	Math.pow(6.0, 2.0) evaluates to 36.0.
abs(x)	Absolute value of x	Math.abs(-99.5) evaluates to 99.5.

Pattern

`Math.methodName(arguments)`

Always identify the class, method, and required argument(s).

```
double a = Math.sqrt(49.0)     // returns 7.0
double b = Math.pow(2.0, 4.0)  // returns 16.0
int c = Math.abs(-12)          // returns 12
```

Common examples #2

A few common methods from the standard Math class.

Method	Behavior	Example
min(x,y)	Returns the smaller value	Math.min(2.0, 16.0) evaluates to 2.0
max(x,y)	Returns the larger value	Math.max(2.0, 16.0) evaluates to 16.0

```
Math.min(49.0, 22.0)   // returns 22.0
Math.max(2.0, 48.0)    // returns 48.0
```

To find the largest of three numbers, you nest one Math.max() call inside another.

```
Math.max(max(x, y), z);//returns the largest number of x,y,z
```

Math Method Return Types

- Math.sqrt(x) → always returns a **double**
- Math.pow(x, y) → always returns a **double**

<< Math.sqrt() and Math.pow() are return double>>

- Math.min(x, y) → return type depends on argument type
- Math.max(x, y) → return type depends on argument type
- Math.abs(x) → return type depends on argument type

<< Math.max(), Math.min(), and Math.abs return type matches arguments' >>

```
1  int x = 10;
2  int y = 20;
3
4  int largest = Math.max(x, y);      // OK
5  double root = Math.sqrt(x);        // OK
6  double power = Math.pow(x, y);     // OK
```

Put it inside a complete program

Practice the full flow: INPUT → PROCESSING → OUTPUT.

```
import java.util.Scanner;

public class Main {
  public static void main(String[] args) {
    Scanner scnr = new Scanner(System.in);

    System.out.print("Please enter a number: ");
    double number = scnr.nextDouble(); // INPUT
    double result = Math.sqrt(number);  // PROCESSING
    System.out.printf("%.2f\n", result);// OUTPUT
  }
}
```



Goal: understand the method call well enough to build the entire program independently.

Quick check

Can you explain each part without looking back?



1. Why is no import required for the Math class?
2. In Math.pow(2.0, 3.0), what are the arguments?
3. What must appear before a Math class method name?

```
Remember: Math. + method name + ( arguments )
```

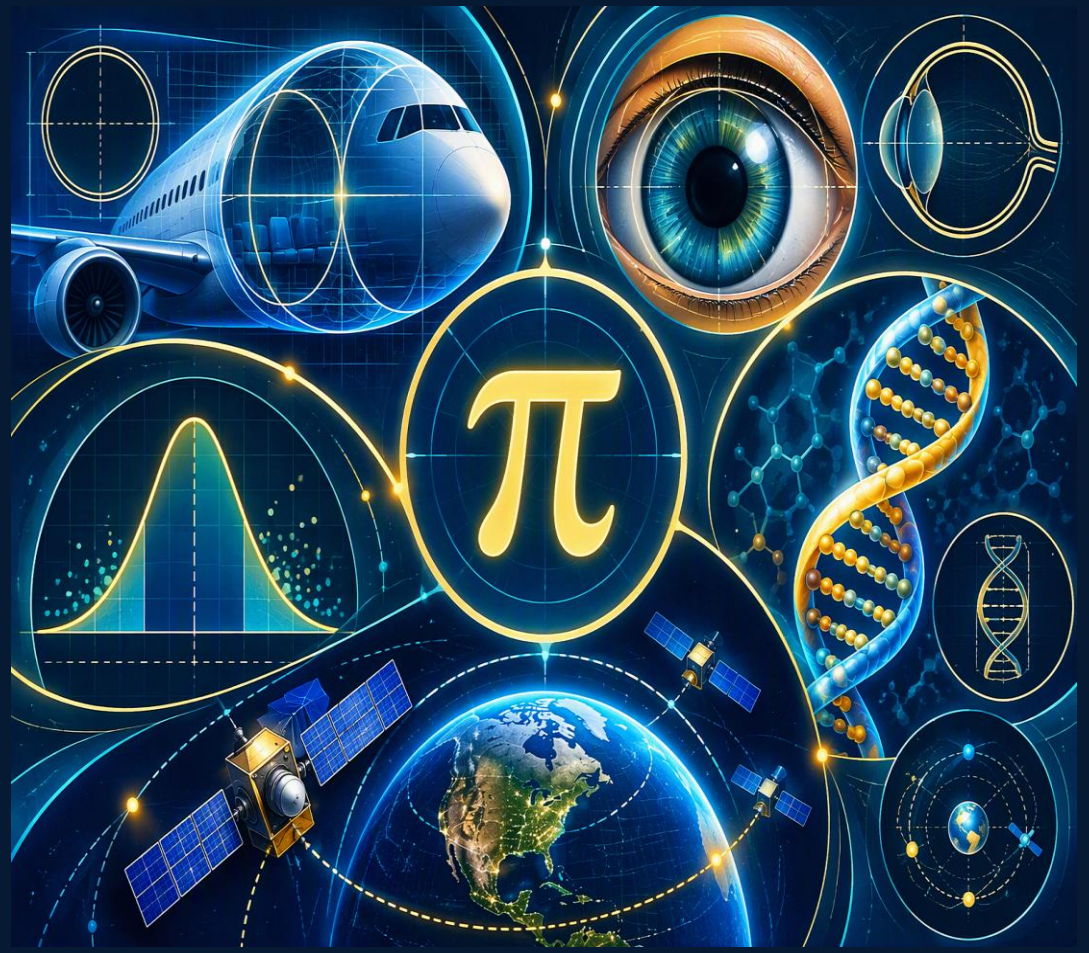
Math.PI: Java’s π constant

Use Math.PI when a calculation requires the mathematical constant pi.

```
Math.PI
```

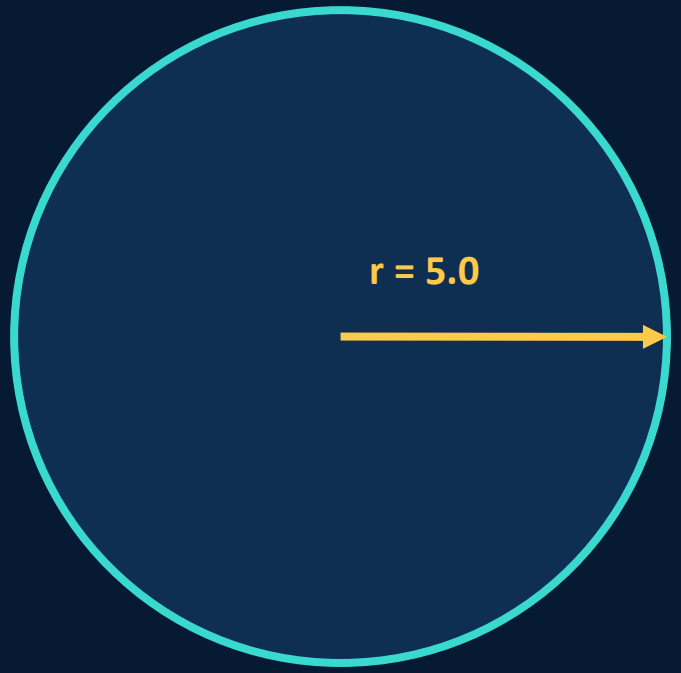
- Provided by the java.lang.Math class
- Type: double
- Approximate value: 3.141592653589793
- It is a constant, not a method, so there are no parentheses.

Write Math.PI, not Math.PI()



Using Math.PI with a circle

Suppose the radius r = 5.0.



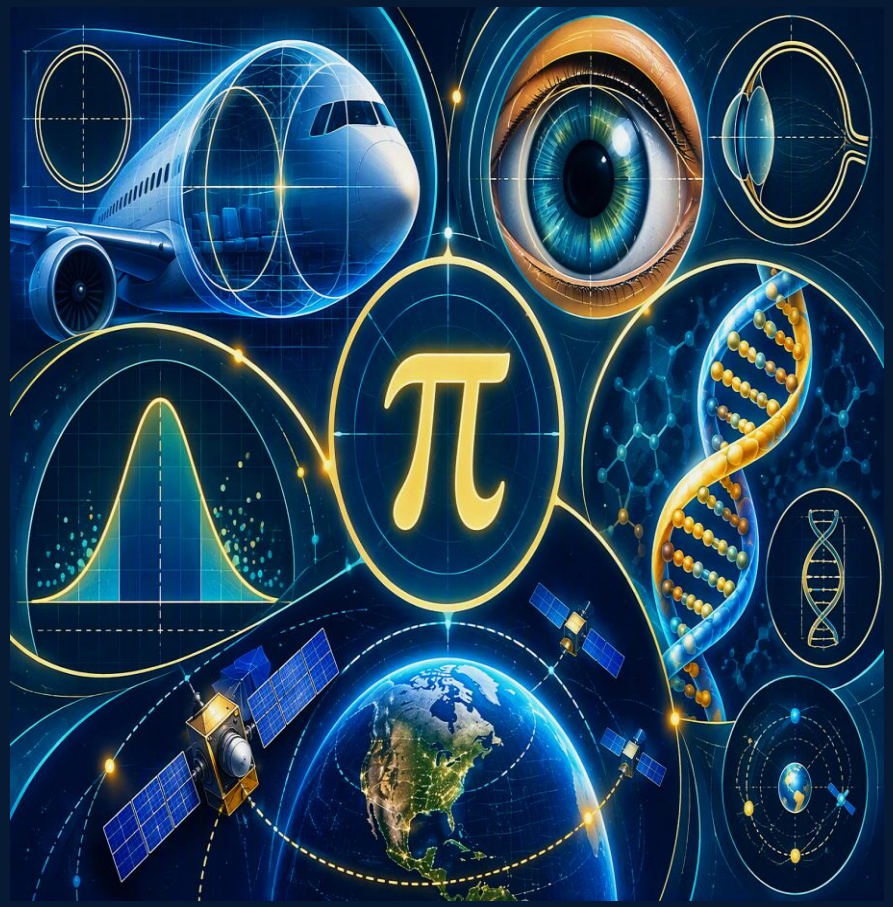

```
double circumference = 2 * Math.PI * r;
double area = Math.PI * Math.pow(r, 2);
```

Circumference = 31.41592653589793
Area = 78.53981633974483

π appears whenever the circle formula calls for it. Math.PI supplies that value directly.

Using Math.PI with a sphere

The same radius, r = 5.0, can be used in 3D formulas.



```
double volume = (4.0 / 3.0) * Math.PI *
Math.pow(r, 3);
double surfaceArea = 4 * Math.PI * Math.pow(r, 2);
```

Volume = 523.5987755982989
Surface Area = 314.1592653589793

Important: use 4.0 / 3.0 so Java performs floating-point division.

Complete Java example

Input → Processing with Math.PI → Output

```
import java.util.Scanner;

public class Main {
  public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    System.out.print("Enter radius: ");
    double r = input.nextDouble();

    double circumference = 2 * Math.PI * r;
    double area = Math.PI * Math.pow(r, 2);

    System.out.printf("Circumference = %.2f%n", circumference);
    System.out.printf("Area = %.2f%n", area);
  }
}
```

Notice

Math.PI is used as a value.

Math.pow(r, 2) is a method call.

Both can be combined in the same expression.

Where π appears beyond the classroom

π is important whenever circular, rotational, periodic, or spherical relationships appear.

Examples to discuss

- Aerospace and engineering geometry
- Medical and scientific modeling
- Biochemistry and molecular research
- Statistics and probability
- Navigation and GPS calculations



Teaching point: Math.PI gives Java programs a precise built-in value of π for formulas that require it.